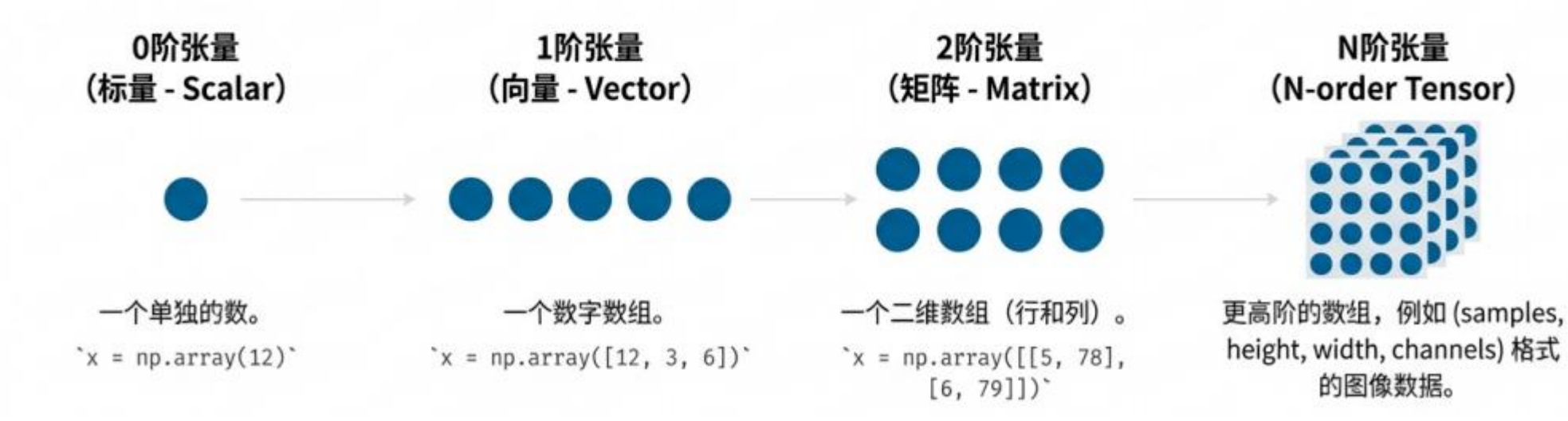


2.1 标量、向量、矩阵和张量

张量是深度学习的基本数据结构，可以看作是多维数组。所有数据——图像、文本、时间序列——最终都会被转换为张量。



2.1 标量、向量、矩阵和张量

- `import torch`
- `import numpy as np`
- `example = torch.tensor(np.arange(24).reshape([2, 3, 4]))`

这个三阶张量是由2个3行4列的矩阵构成

```
tensor([[[ 0,  1,  2,  3],
         [ 4,  5,  6,  7],
         [ 8,  9, 10, 11]],
        [[12, 13, 14, 15],
         [16, 17, 18, 19],
         [20, 21, 22, 23]]], dtype=torch.int32)
```

`example2 = example.transpose(0, 1)`

```
tensor([[[ 0,  1,  2,  3],
         [12, 13, 14, 15]],
        [[ 4,  5,  6,  7],
         [16, 17, 18, 19]],
        [[ 8,  9, 10, 11],
         [20, 21, 22, 23]]], dtype=torch.int32)
```

2.1 标量、向量、矩阵和张量

example2 = example.transpose(0, 2)

```
tensor([[[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]],

        [[12, 13, 14, 15],
          [16, 17, 18, 19],
          [20, 21, 22, 23]]], dtype=torch.int32)
```

```
tensor([[[ 0, 12],
          [ 4, 16],
          [ 8, 20]],

        [[ 1, 13],
          [ 5, 17],
          [ 9, 21]],

        [[ 2, 14],
          [ 6, 18],
          [10, 22]],

        [[ 3, 15],
          [ 7, 19],
          [11, 23]]], dtype=torch.int32)
```

2.2 矩阵和向量相乘

- torch.mm

实现**矩阵乘法**。A的维度如果是 $m \times n$ ，B的维度如果是 $n \times p$ ，则`torch.mm(A,B)`的维度是 $m \times p$ 。此处A，B只能是二维的张量

```
mat1 = torch.arange(12).reshape(3,4)
```

```
mat2 = torch.arange(12).reshape(4,3)
```

```
torch.mm(mat1, mat2)
```

```
tensor([[ 42,  48,  54],
```

```
        [114, 136, 158],
```

```
        [186, 224, 262]])
```


2.2 矩阵和向量相乘

- torch.mul

实现的是**按元素乘**。如果矩阵a, b的维度不同, 会按照广播机制先分别将a, b的数据进行扩充, 如果能够扩充到同维度, 再进行元素乘。

```
import torch
a = torch.ones(3,4)
print(a)
b = torch.Tensor([1,2,3]).reshape((3,1))
print(b)
print(torch.mul(a, b))
```

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
tensor([[1.],
        [2.],
        [3.]])
tensor([[1., 1., 1., 1.],
        [2., 2., 2., 2.],
        [3., 3., 3., 3.]])
```

2.2 矩阵和向量相乘

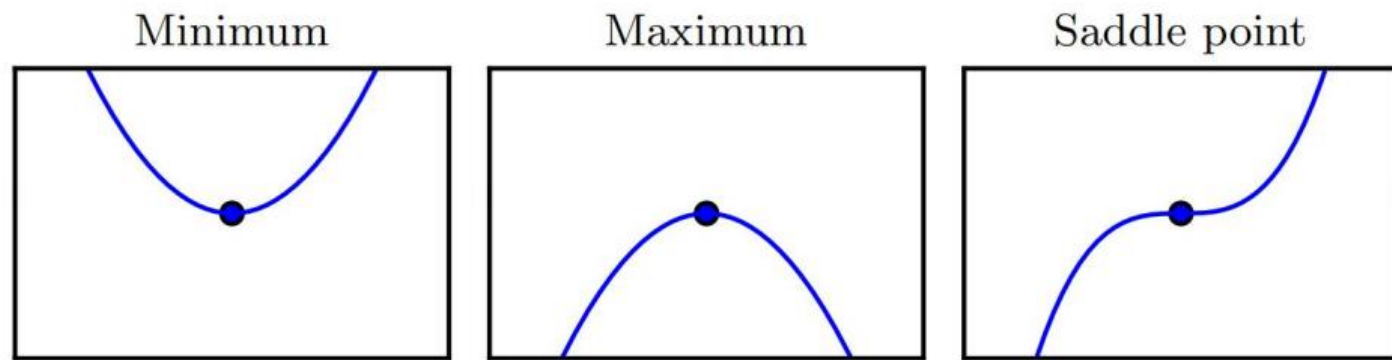
- `torch.dot`

点积。A是维度m的向量，B是维度m的向量。`torch.dot(A,B)`返回按元素乘后的求和结果，即点积结果。注意此处要求A，B均为1D的张量。

```
torch.dot(torch.tensor([2, 3]), torch.tensor([2, 1]))
```

4.3 基于梯度的优化方法

当 $f'(x) = 0$ ，导数无法提供往哪个方向移动的信息。 $f'(x) = 0$ 的点称为 **临界点**（critical point）或 **驻点**（stationary point）。一个 **局部极小点**（local minimum）意味着这个点的 $f(x)$ 小于所有邻近点，因此不可能通过移动无穷小的步长来减小 $f(x)$ 。一个 **局部极大点**（local maximum）意味着这个点的 $f(x)$ 大于所有邻近点，因此不可能通过移动无穷小的步长来增大 $f(x)$ 。有些临界点既不是最小点也不是最大点。这些点被称为 **鞍点**（saddle point）。



4.3 基于梯度的优化方法

- 梯度下降法 (gradient descent, GD) : 沿着负梯度方向 (即下山方向) 更新模型参数

$$x^{k+1} = x^k - \eta \nabla f(x^k)$$

- x^k : 模型第 k 次迭代的参数值
- η : 学习率 (也称为步长)
- $\nabla f(x^k)$: 梯度
- x^0 : 初始值
- $\|\nabla f(x^k)\| \leq \varepsilon$: 终止条件

算法 3.1 梯度下降法

1. 初始化变量 x^0
2. For $k = 0, 1, 2, \dots, T$
3. 在当前迭代点 x^k 处计算梯度 $\nabla f(x^k)$
4. 选择步长 η
5. 更新模型参数: $x^{k+1} = x^k - \eta \nabla f(x^k)$
6. If $\|\nabla f(x^k)\| \leq \varepsilon$
7. break
8. End if
9. End for

6.3 过拟合与欠拟合

泛化误差 vs. 经验误差

泛化误差：在“未来”样本上的误差

经验误差：在训练集上的误差，亦称“训练误差”

泛化误差越小越好

经验误差是否越小越好？

NO! 因为会出现“**过拟合**” (overfitting)

6.3 过拟合与欠拟合

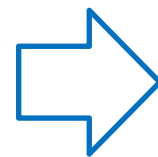
- 决定ML/DL算法效果好坏的因素:

1. 训练误差高、测试误差高



欠拟合

2. 训练误差低、测试误差误差高



过拟合

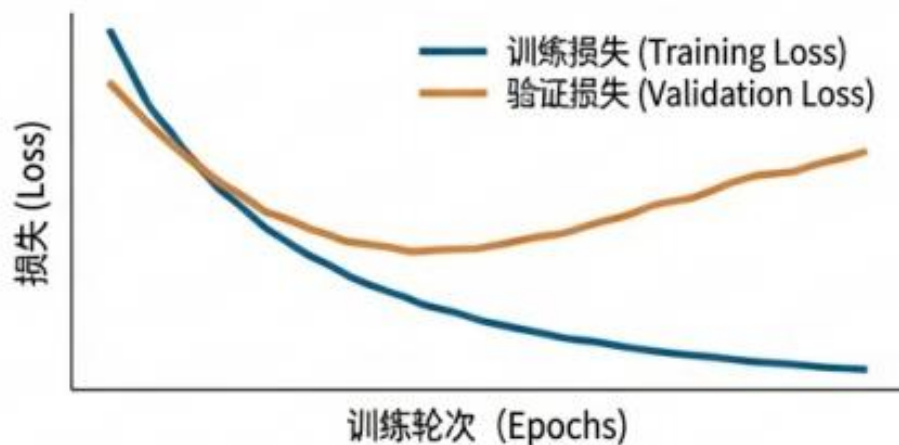
- 欠拟合是指模型不能在训练集上获得足够低的误差;
- 过拟合是指训练误差虽然低, 但是测试误差很高。

6.3 过拟合与欠拟合

过拟合 (Overfitting)

Symptom

模型在训练集上表现良好，但在测试集上表现较差。
(Exam Q8)



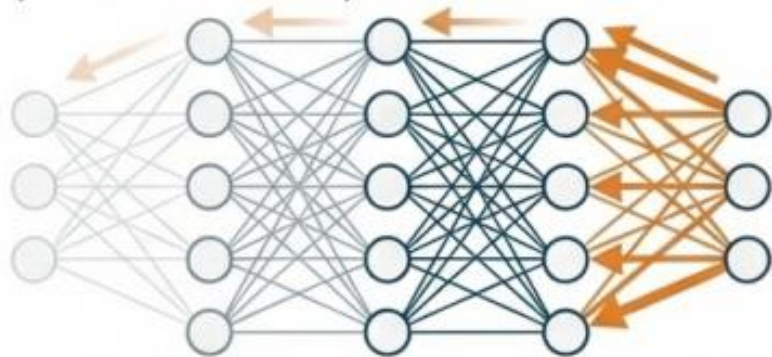
解决方案 (Solutions)

- 增加训练数据 / 数据增强
- 使用正则化 (Use Regularization)
- 采用 Dropout
- 降低模型复杂度

梯度消失 (Vanishing Gradients)

Root Cause

在深层网络中，反向传播基于链式法则。使用导数小于1的激活函数（如Sigmoid）时，导数连乘会导致梯度呈指数级衰减，使靠近输入层的参数几乎无法更新。(Exam Short Ans 2)



解决方案 (Solutions)

- 使用非饱和激活函数，如 ReLU ($f(x) = \max(0, x)$)
- 使用批归一化
- 使用残差连接

7 深度前馈神经网络

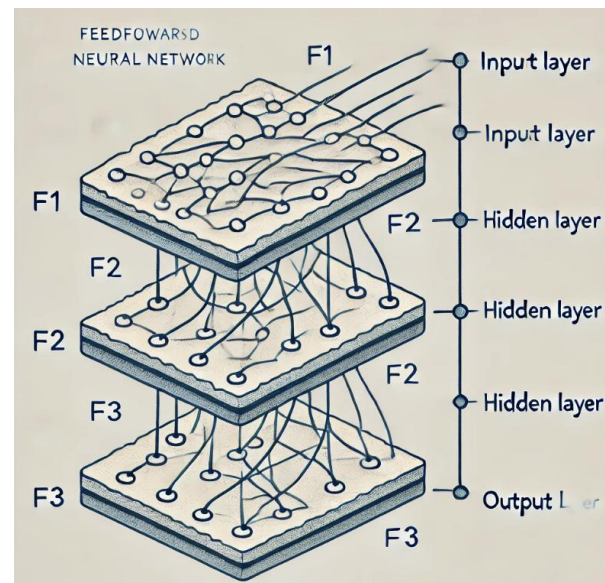
12

深度前馈网络 (deep feedforward network), 也叫作 前馈神经网络 (feedforward neural network) 或者 多层感知机 (multilayer perceptron, MLP), 是典型的深度学习模型。前馈网络的目标是近似某个函数 f^* 。例如, 对于分类器, $y = f^*(\mathbf{x})$ 将输入 $\mathbf{x}$ 映射到一个类别 y 。前馈网络定义了一个映射 $\mathbf{y} = f(\mathbf{x}; \boldsymbol{\theta})$, 并且学习参数 $\boldsymbol{\theta}$ 的值, 使它能够得到最佳的函数近似。

7 深度前馈神经网络

13

前馈神经网络被称作网络（network）是因为它们通常用许多不同函数复合在一起来表示。该模型与一个有向无环图相关联，而图描述了函数是如何复合在一起的。例如，我们有三个函数 $f^{(1)}$, $f^{(2)}$ 和 $f^{(3)}$ 连接在一个链上以形成 $f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$ 。这些链式结构是神经网络中最常用的结构。在这种情况下， $f^{(1)}$ 被称为网络的第一层（first layer）， $f^{(2)}$ 被称为第二层（second layer），以此类推。链的全长称为模型的深度（depth）。



7 深度前馈神经网络

14

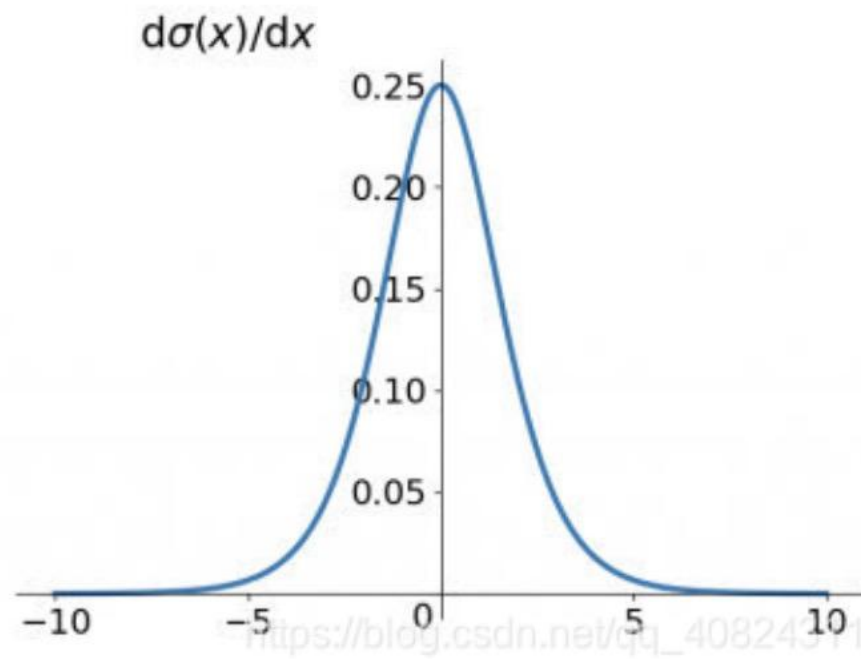
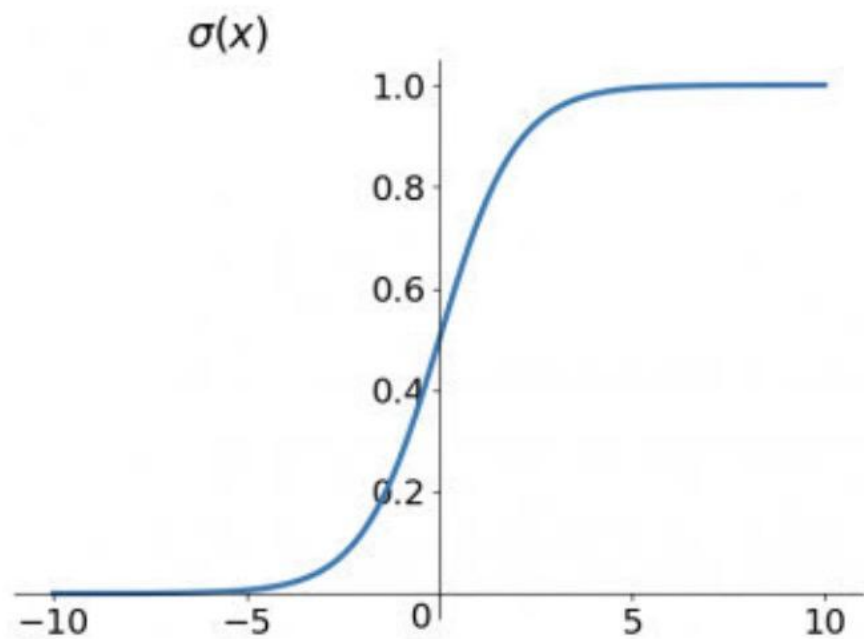
前馈网络的最后一层被称为**输出层** (output layer)。在神经网络训练的过程中，我们让 $f(\mathbf{x})$ 去匹配 $f^*(\mathbf{x})$ 的值。训练数据为我们提供了在不同训练点上取值的、含有噪声的 $f^*(\mathbf{x})$ 的近似实例。每个样本 $\mathbf{x}$ 都伴随着一个标签 $y \approx f^*(\mathbf{x})$ 。训练样本直接指明了输出层在每一点 $\mathbf{x}$ 上必须做什么；它必须产生一个接近 y 的值。但是训练数据并没有直接指明其他层应该怎么做。学习算法必须决定如何使用这些层来产生想要的输出，但是训练数据并没有说每个单独的层应该做什么。相反，学习算法必须决定如何使用这些层来最好地实现 f^* 的近似。因为训练数据并没有给出这些层中的每一层所需的输出，所以这些层被称为**隐藏层** (hidden layer)。

7.2.1 代价函数

15

饱和、梯度消失

$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$ ，其导数为 $\text{sigmoid}(x) = \frac{e^{-x}}{(1+e^{-x})^2}$ ，图像如下：

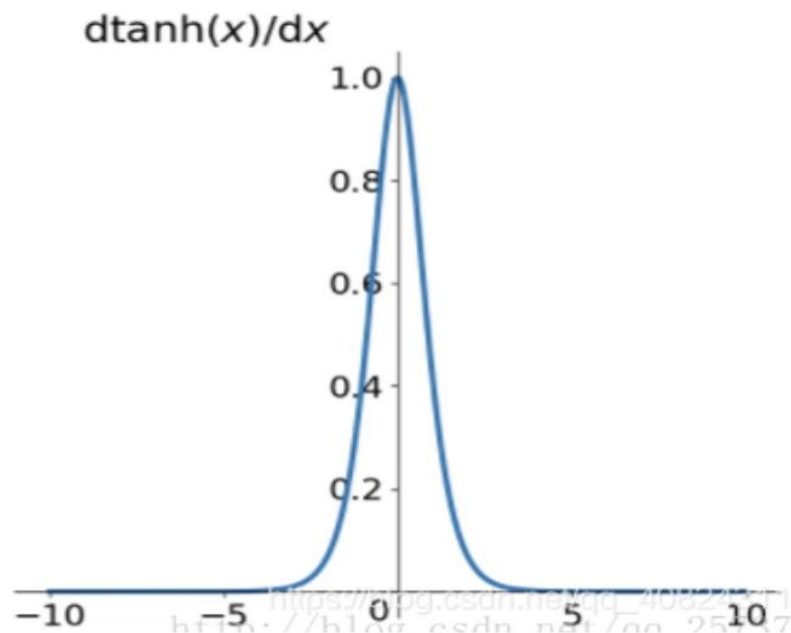
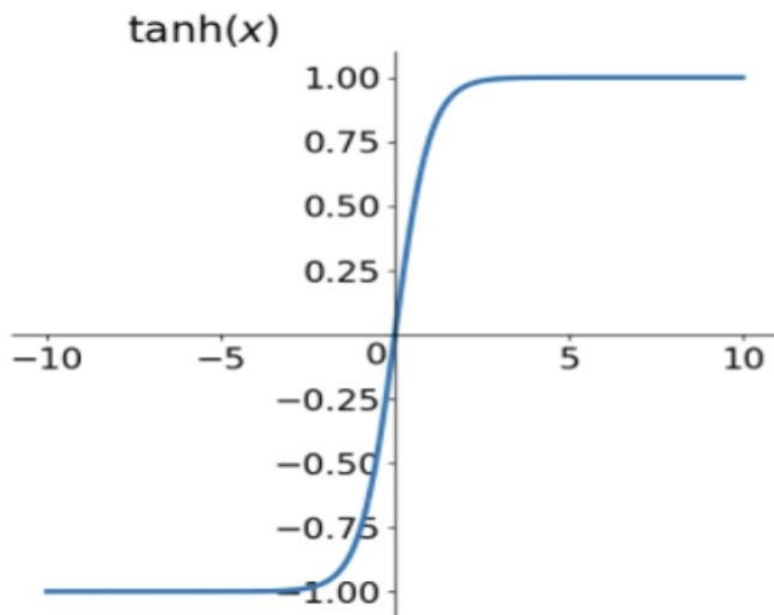


7.2.1 代价函数

16

饱和、梯度消失

$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ ，其导数为 $\frac{4e^{2x}}{(e^{2x} + 1)^2}$ ，图像如下：



可以看出，相较于Sigmoid函数有所改善，但导数仍小于1，不能避免梯度消失的情况。

7.2.1 代价函数

17

□ 常见的激活函数：

- **ReLU函数**：深度神经网络中较为常用的激活函数。当 $x > 0$ 时，导数为1，缓解了梯度爆炸和梯度消失问题。然而当 $x < 0$ 时，ReLU在反向传播过程中梯度为0，导致神经元不能被激活。

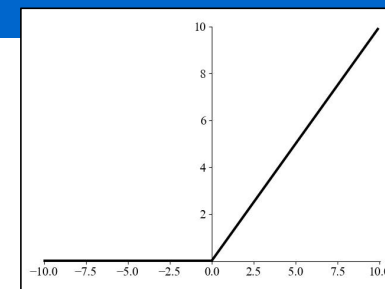
$$\text{ReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- **Leaky ReLU函数**：是基于ReLU函数的改进版本，对于 $x < 0$ ，Leaky ReLU具有一个很小的梯度 γ ，使得神经元可以被激活。一般情况下， γ 采用一个很小的常数，如0.01。

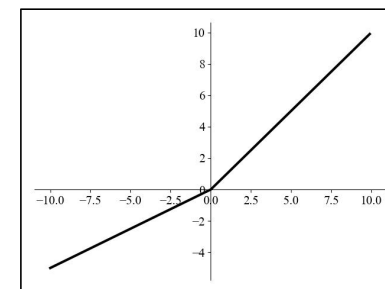
$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0 \\ \gamma x, & x < 0 \end{cases}$$

- **Swish函数**：是一种自门控激活函数，采用Sigmoid函数作为一种门控机制。当 x 较大时， $\sigma(x)$ 趋向于1，Swish函数结果近似于 x 本身，此时Swish函数近似为ReLU函数；当 x 较小时， $\sigma(x)$ 趋向于0，Swish函数结果近似于0。由于Swish激活函数的函数较为复杂，在训练过程中较为耗时。

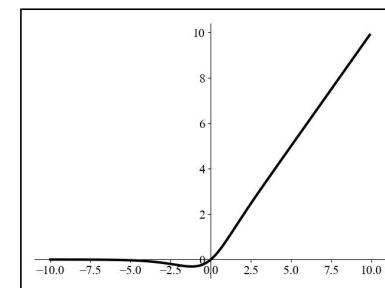
$$\text{swish}(x) = x\sigma(x)$$



(d) ReLU函数



(e) Leaky ReLU函数



(f) Swish函数

7.3.1 ReLU及其扩展

18

整流线性单元使用激活函数 $g(z) = \max\{0, z\}$ 。

整流线性单元易于优化，因为它们和线性单元非常类似。线性单元和整流线性单元的唯一区别在于整流线性单元在其一半的定义域上输出为零。这使得只要整流线性单元处于激活状态，它的导数都能保持较大。它的梯度不仅大而且一致。整流操作的二阶导数几乎处处为 0，并且在整流线性单元处于激活状态时，它的一阶导数处处为 1。这意味着相比于引入二阶效应的激活函数来说，它的梯度方向对于学习来说更加有用。

7.4 架构设计

19

大多数神经网络被组织成称为层的单元组。大多数神经网络架构将这些层布置成链式结构，其中每一层都是前一层的函数。在这种结构中，第一层由下式给出：

$$\mathbf{h}^{(1)} = g^{(1)} \left(\mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)} \right);$$

第二层由

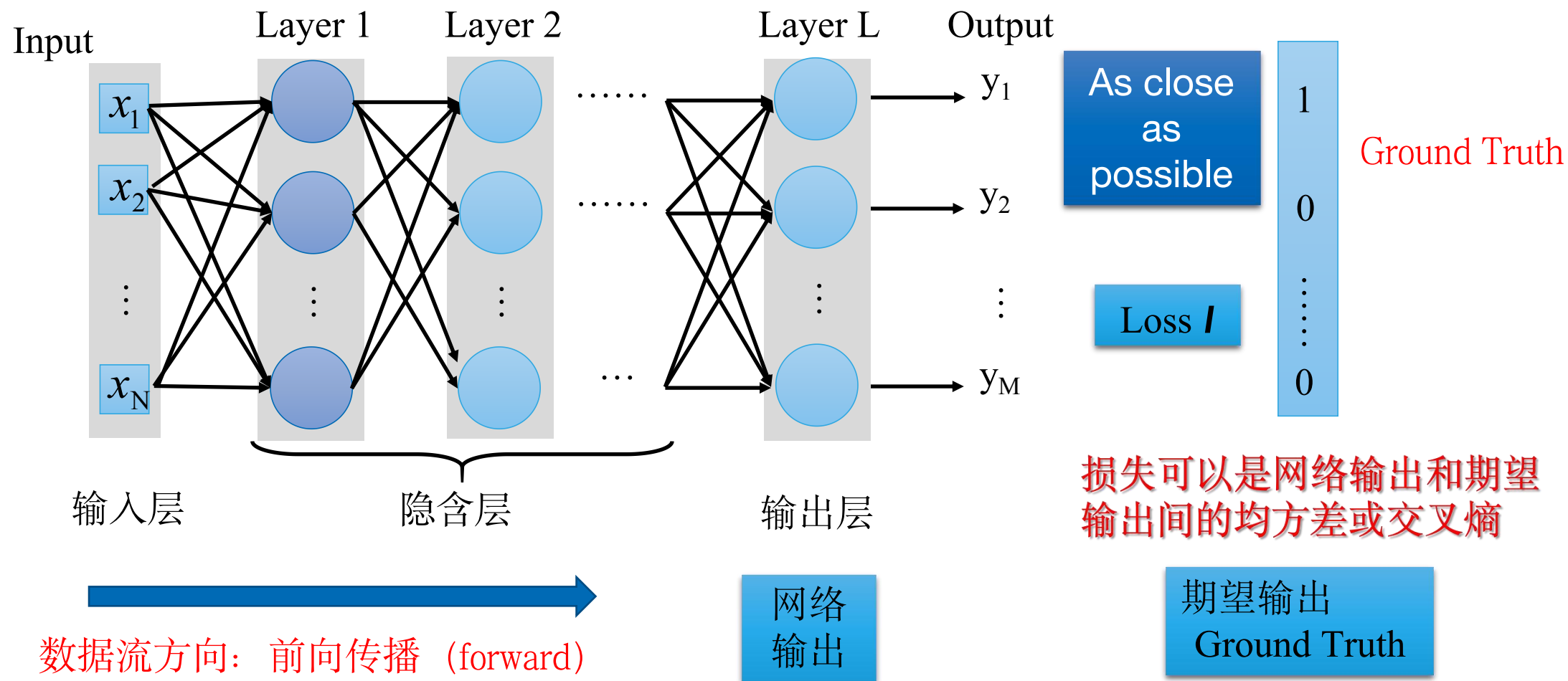
$$\mathbf{h}^{(2)} = g^{(2)} \left(\mathbf{W}^{(2)\top} \mathbf{h}^{(1)} + \mathbf{b}^{(2)} \right);$$

给出，以此类推。

7.4 架构设计

20

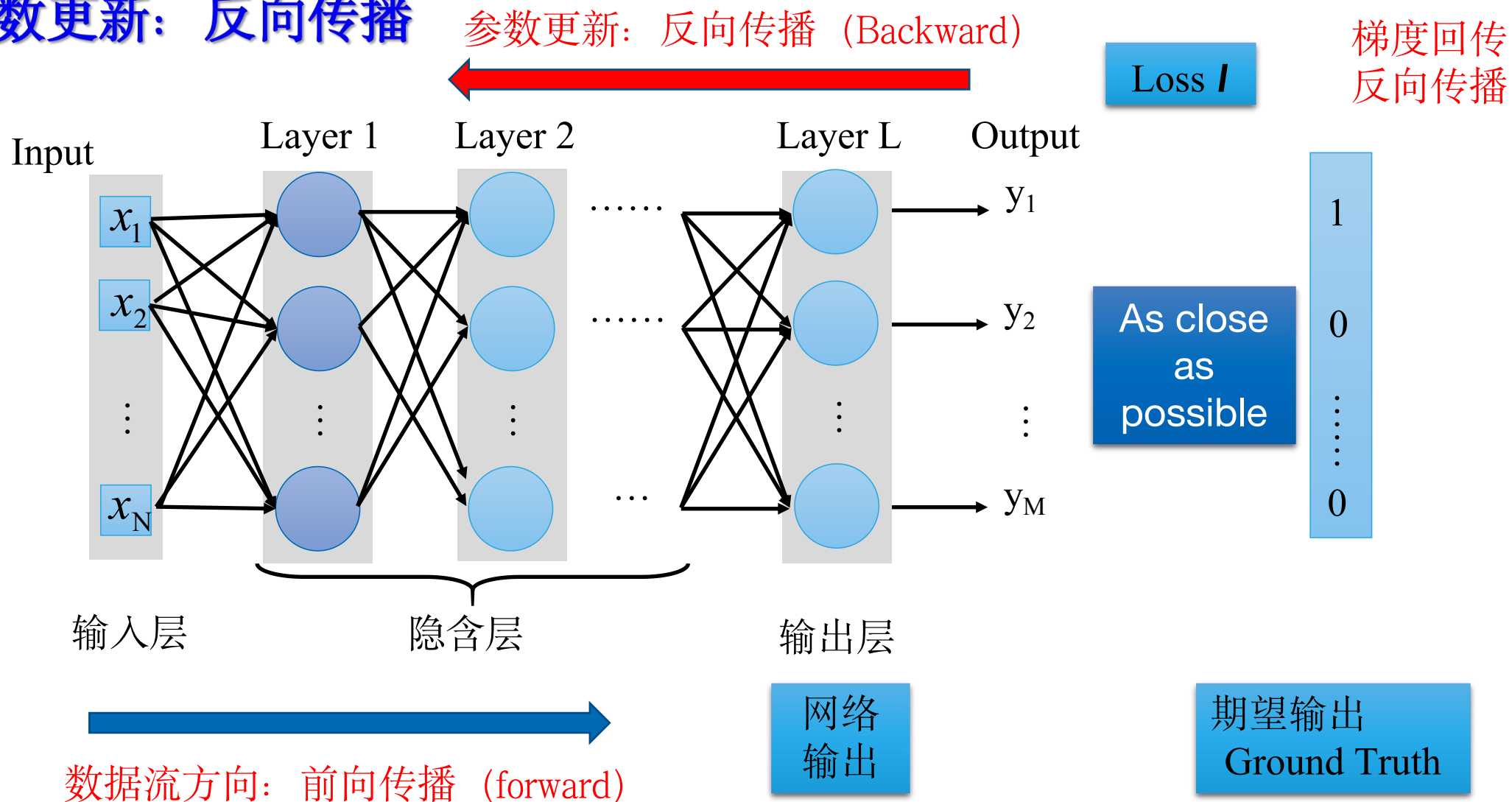
□ 损失函数:



7.4 架构设计

21

参数更新: 反向传播



8.1 卷积运算

22

$$y(n) = \sum_{m=-\infty}^{\infty} x(m)h(n-m) = x(n) * h(n)$$

计算步骤如下：

(1) **翻褶**：先在坐标轴 m 上画出 $x(m)$ 和 $h(m)$ ，将 $h(m)$ 以纵坐标为对称轴折叠成 $h(-m)$ 。

(2) **移位**：将 $h(-m)$ 移位 n ，得 $h(n-m)$ 。当 n 为正数时，右移 n ；当 n 为负数时，左移 n 。

(3) **相乘**：将 $h(n-m)$ 和 $x(m)$ 的对应序列值相乘。

(4) **相加**：把所有的乘积累加起来，即得 $y(n)$ 。

8.3 卷积步长

23

卷积运算在水平和垂直方向单次滑动的像素距离称为卷积步长，一般用 s 表示。

$$n \times n * f \times f \longrightarrow (n + 2p - f + 1) \times (n + 2p - f + 1)$$

$$\begin{matrix} \text{stride} = s \\ \text{padding} = p \end{matrix} \longrightarrow \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

8.3 卷积步长

24

Input (4,4) Kernel(3,3) Padding=0 Stride=1
Output = $(4 - 3)/1 + 1 = 2$

Input (5,5) Kernel(3,3) Padding=1 Stride=1
Output = $(5 + 2*1 - 3)/1 + 1 = 5$

Input (6,6) Kernel(3,3) Padding=1 Stride=2
Output = $(6 + 2*1 - 3)/2 + 1 = 2.5 + 1 = 3.5$

8.3 卷积步长

25

- 输入图片的尺寸统一为 $227 \times 227 \times 3$ (高度 $\times$ 宽度 $\times$ 颜色通道数) ,
- 本层一共具有96个卷积核,
- 每个卷积核的尺寸都是 $11 \times 11 \times 3$ 。
- 已知 $\text{stride} = 4$, $\text{padding} = 0$,
- 则输出的高度/宽度为

8.4 三维卷积

26

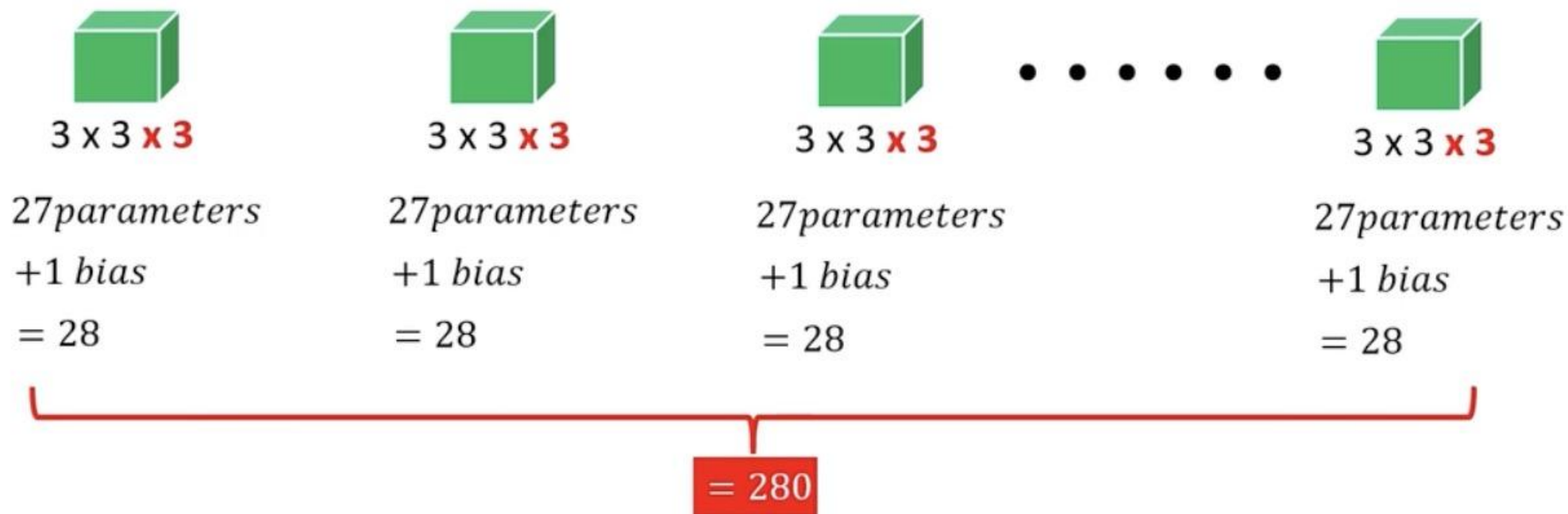
三维卷积的基本特征概括如下：

- (1) 输入图像拥有RGB三个通道，**过滤器的通道数必须与输入图像的通道数相同**。
- (2) 过滤器的通道与输入图像的通道**一一对应**，过滤器在对应图层上做二维卷积。
- (3) 3个RGB通道图层**各自与过滤器做二维卷积**，然后**求和**后算作一次三维卷积，**输出的目标特征图只有一个图层**。

8.5 卷积层

27

假定卷积层采用10个 $3 \times 3 \times 3$ 的过滤器做卷积运算，那么这个卷积层将有多少个参数？



练习题

28

假设输入是一张300x300彩色图像，第一个隐藏层使用了 100 个5*5卷积核做卷积操作，这个隐藏层有多少个参数（包括偏置参数）？

A.7601

B.7600

C. 7500

D.2501

8.7 池化层

29

卷积层
(Conv)

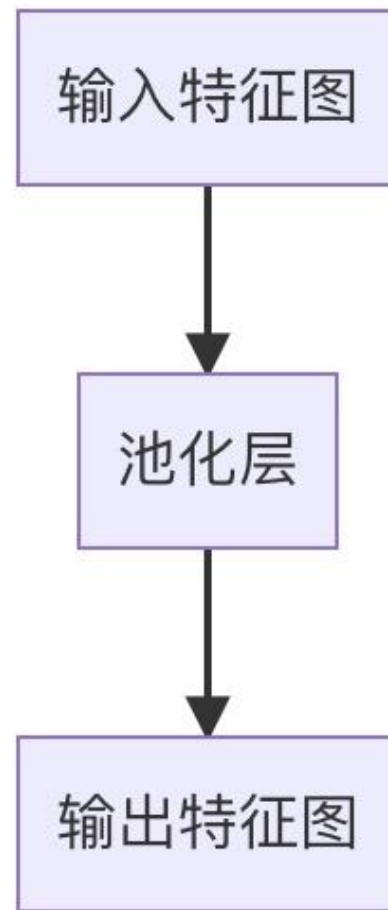
池化层
(Pooling)

全连接层
(FC)

8.7 池化层

30

- 池化层的结构与卷积层类似，它也由多个滤波器组成，每个滤波器对输入数据进行卷积操作，得到一个输出特征图。
- 不同的是，池化层的卷积操作通常不使用权重参数，而是使用一种固定的池化函数，例如**最大池化**、**平均池化**等。
- 池化层主要用于减少数据量和计算复杂度，同时保留重要的特征信息。它可以提高模型的训练速度和泛化能力，避免过拟合等问题。



8.7 池化层

31

f : filter的高和宽

s : stride

Max or Average Pooling

$$f = 2, \quad s = 2$$

目标图像高度和宽度各减一半

$$n_H \times n_W \times n_C$$

$$\left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor$$

8.8 经典卷积神经网络

32

卷积：

- ✓ 局部特征提取
- ✓ 训练中进行参数学习
- ✓ 每个卷积核提取特定模式的特征

池化（下采样）：

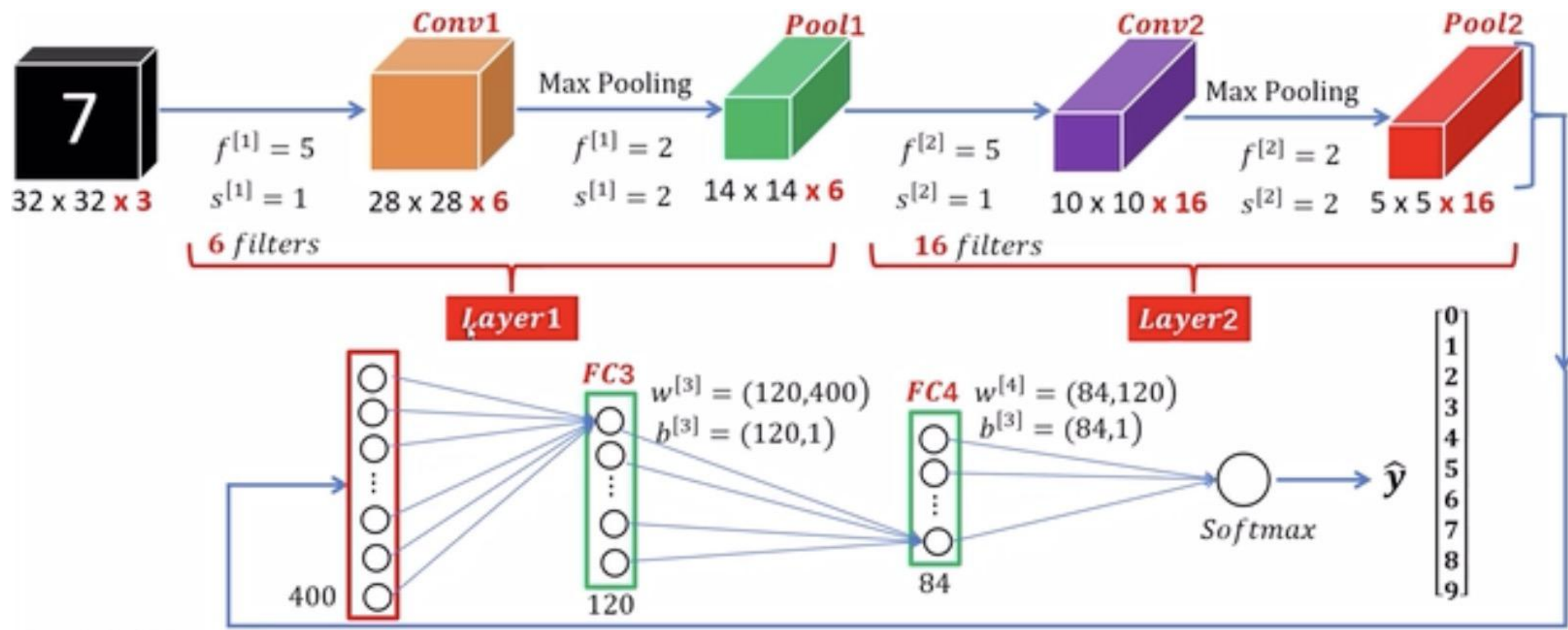
- ✓ 降低数据维度，避免过拟合
- ✓ 增强局部感受野
- ✓ 提高平移不变性

全连接：

- ✓ 特征提取到分类的桥梁

8.8 经典卷积神经网络

33



8.8 经典卷积神经网络

34

网络各层	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072	0
CONV1 (f=5, s=1, filters=6)	(28,28,6)	4,704	456
POOL1(f=2, s=2)	(14,14,6)	1,176	0
CONV2 (f=5, s=1, filters=16)	(10,10,16)	1,600	2,416
POOL2(f=2, s=2)	(5,5,16)	400	0
FC3	(120,1)	120	48,120
FC4	(84,1)	84	10,164
Softmax	(10,1)	10	850
合计			61,795

8.8 经典神经网络

35

深度学习，神经网络越深越好吗？

- 梯度消失、梯度爆炸
- 过拟合
- 退化
- 在反向传播过程中需要对激活函数进行求导，如果导数大于1，那么随着网络层数的增加梯度更新将会朝着指数爆炸的方式增加这就是**梯度爆炸**。
- 同样如果导数小于1，那么随着网络层数的增加梯度更新信息会朝着指数衰减的方式减少，这就是**梯度消失**。
- 因此，梯度消失、爆炸，其**根本原因在于反向传播链式法则**，属于先天不足

9.1 生成对抗网络

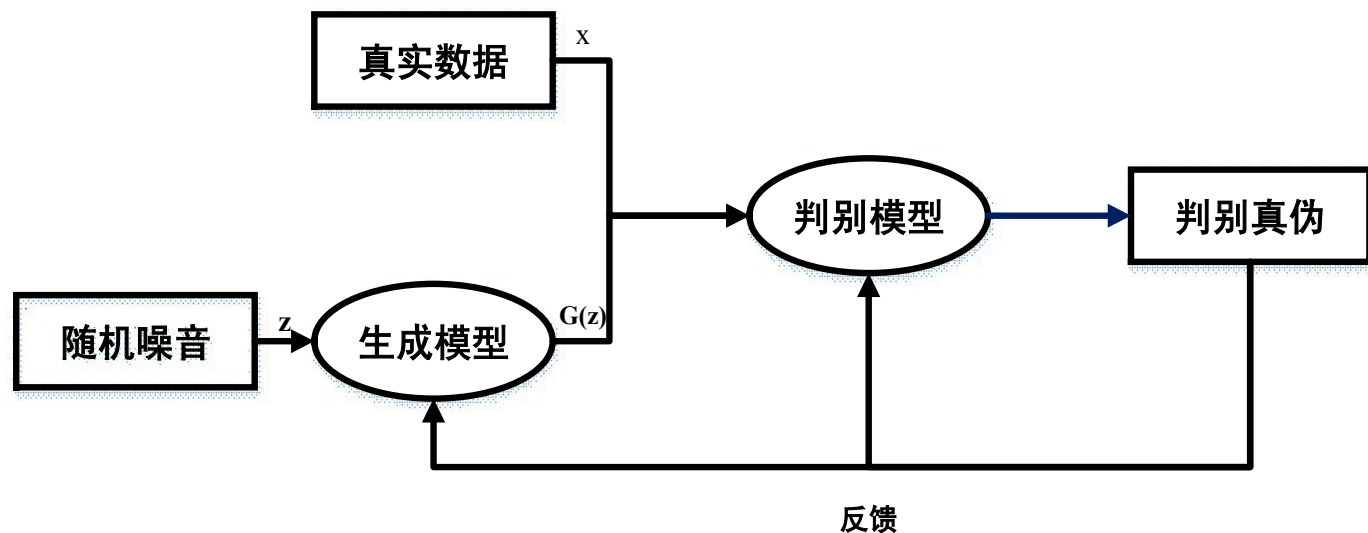
9.1.1 GAN原理

生成式对抗网络的基本原理为：首先根据随机噪声利用生成器生成新图像，然后利用判别器判别一幅图像的真实性（如图像的判别概率为1则为真、为0则为假）；在训练过程中，生成器尽量生成真实图像以欺骗判别器，而判别器则尽量把生成器生成的图像与真实图像分开，两者进而构成了一个动态的“博弈过程”。最终，生成器可生成“以假乱真”的图像（判别器难以进行判别或判别概率为1）。

9.1 生成对抗网络

9.1.1 GAN原理

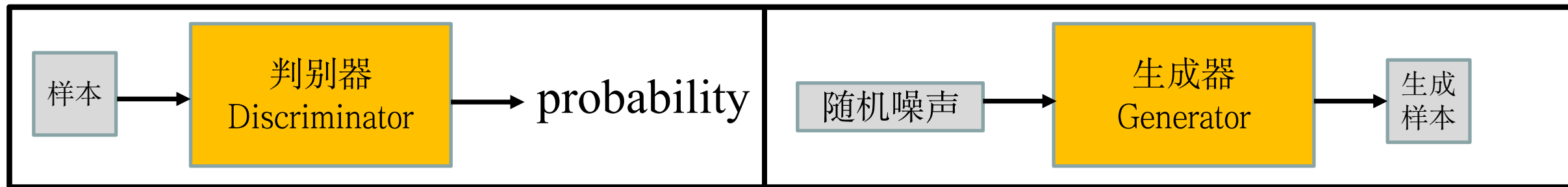
- 生成器 (Generator - G) : 目标是学习真实数据的分布, 从随机噪声中创造出“以假乱真”的样本。
- 判别器 (Discriminator - D) : 目标是判断一个给定的样本是“真实的”还是由生成器“伪造的”。



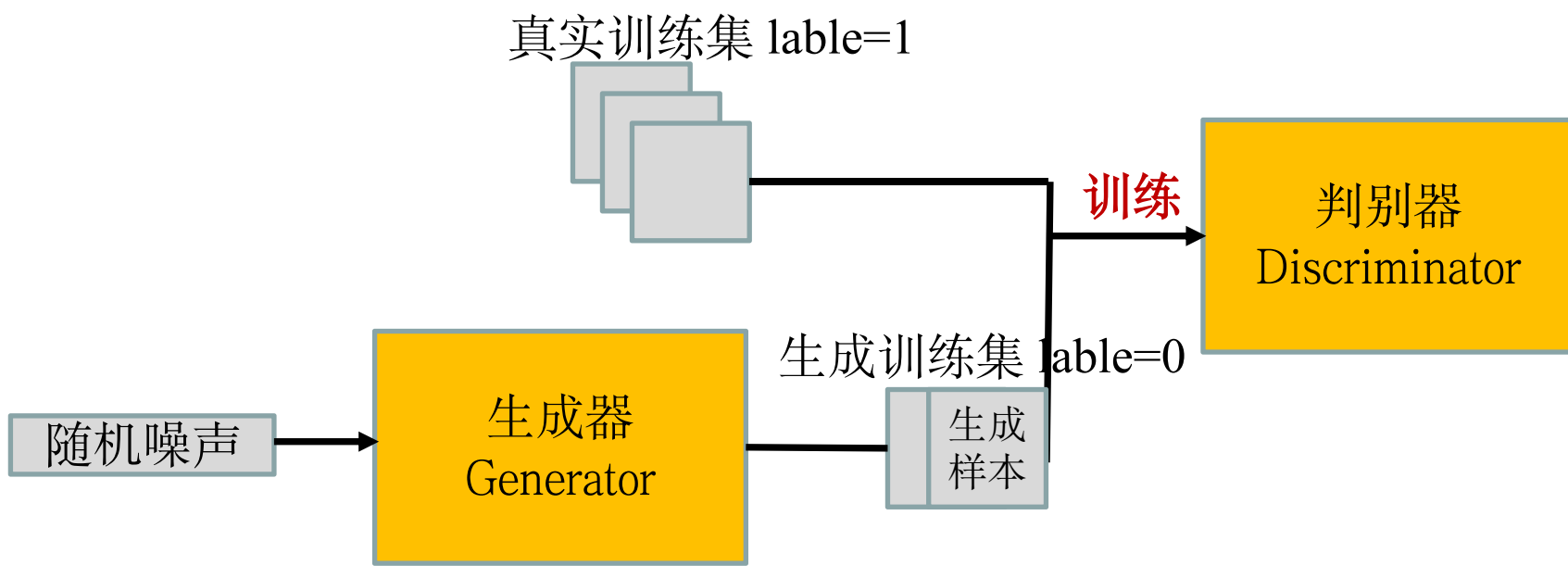
9.1 生成对抗网络

9.1.1 GAN原理

判别器训练



1. 训练判别器D (警察)



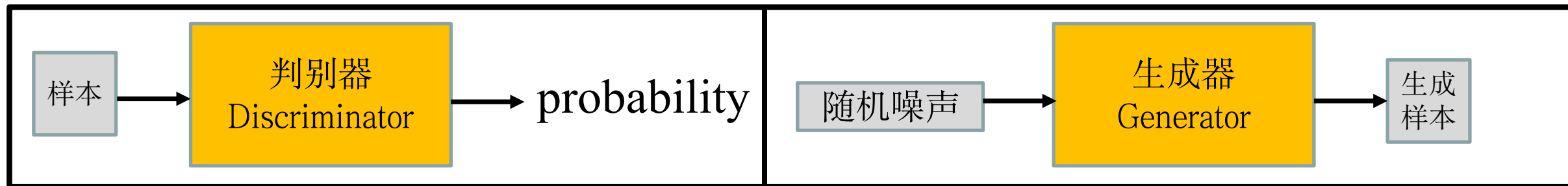
判别器D的训练集由两部分组成

- ①人为的贴标签为1的真实训练集
- ②人为的贴标签为0的生成器生成的假样本

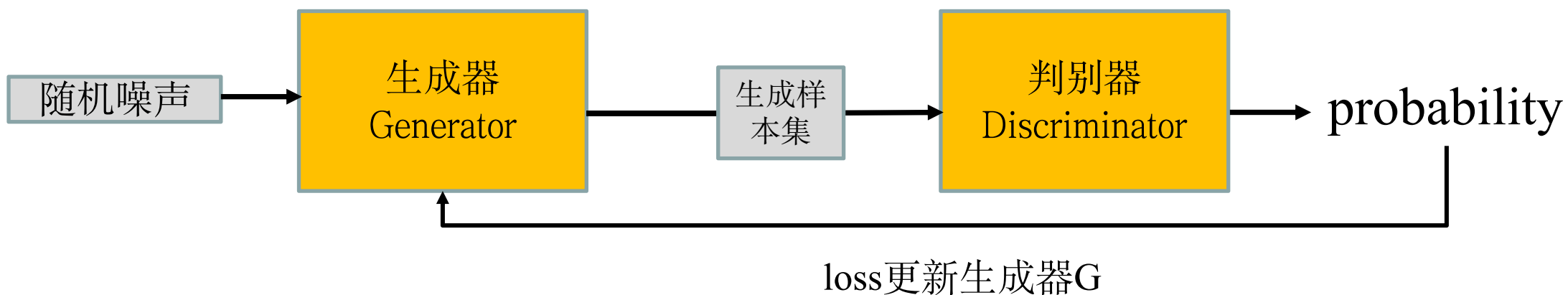
9.1 生成对抗网络

9.1.1 GAN原理

生成器训练



2. 训练生成器G (小偷培训方法)



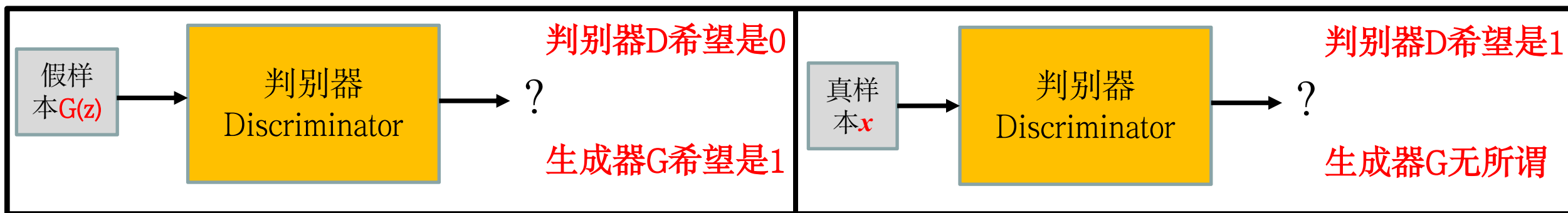
9.1 生成对抗网络

9.1.1 GAN原理

GAN的优化函数

我们已经基本知道GAN中生成器G和判别器D各自的输入及其输出，

我们假设判别器D对给定样本的输出概率值越接近1表示判别器D越坚信其属于真样本，越接近0则表示判别器D越坚信其属于假样本。



生成器G希望其生成的假样本输入给判别器D时,判别器能给出接近1的值;

判别器D希望能够正确的分别假样本和真样本。

9.1 生成对抗网络

9.1.2 实现框架

步骤1: G接收一个随机的噪声 z 并生成数据，D判断生成的数据是不是“真实的”。

步骤2: 计算生成模型G的损失函数。其中， $G(z)$ 表示生成模型的输出， $D(G(z))$ 表示判别模型D判断生成模型G生成的数据是否为真实的概率。

步骤3: 计算判别模型D的损失函数。其中， x 表示输入参数，即真实样本数据， $D(x)$ 表示判别模型的输出，即输入 x 为真实数据的概率。

步骤4: 计算优化函数

$$\min_G \max_D \text{Loss}(D, G) = \mathbb{E}_{X \sim f_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{Z \sim f_z(z)} [\log(1 - D(G(z)))]$$

9.1 生成对抗网络

9.1.2 实现框架

步骤5: 在训练过程当中，G尽可能生成真实的数据去欺骗D，而D则尽量把G生成的数据和真实的数据区分开来，最终生成模型和判别模型形成了一个动态的“博弈过程”。

步骤6: 判断D是否能判别G所生成的数据为真实，若能，则得到训练好的数据，否则，需要调整输入参数 x 后继续执行步骤2。

步骤7: 理想状态下，G生成足以“以假乱真”的 $G(z)$ ，对于D而言，无法判定G所生成的数据是否真实，即 $D(G(z))=0.5$ 。

9.1 生成对抗网络

9.1.2 实现框架

步骤5: 在训练过程当中，G尽可能生成真实的数据去欺骗D，而D则尽量把G生成的数据和真实的数据区分开来，最终生成模型和判别模型形成了一个动态的“博弈过程”。

步骤6: 判断D是否能判别G所生成的数据为真实，若能，则得到训练好的数据，否则，需要调整输入参数x后继续执行步骤2。

步骤7: 理想状态下，G生成足以“以假乱真”的 $G(z)$ ，对于D而言，无法判定G所生成的数据是否真实，即 $D(G(z))=0.5$ 。

超越从零开始：如何通过迁移学习站在巨人肩膀上？

迁移学习是将一个在大型数据集（如 ImageNet）上预训练好的模型的知识，迁移应用到另一个相关但数据量较小的任务上。

Why It Works: 预训练模型已经学习到了通用的底层和中层特征（如边缘、纹理、形状）。这些特征对于许多视觉任务都是有用的。我们只需在新任务上对模型的高层特征和分类器进行微调（fine-tuning）。

1. ****加载预训练模型**:**

```
`model = models.resnet18(pretrained=True)`
```

2. ****冻结底层参数**:** 将早期卷积层的参数设置为不可训练。这些是通用特征提取器。

3. ****替换顶层分类器**:** 移除原有的全连接层，换上一个适用于新任务（如不同类别数量）的新分类器。

4. ****在少量新数据上微调**:** 只训练新的分类器和部分解冻的高层网络参数。

